

Neural Networks**Overview (what we will cover)**

16 Feb 2026

- modern neural networks
 1. are not new
 2. not all that different from previously discussed machine learning models
 3. rely on function compositions to learn useful features
 4. have a hierarchical architecture
 5. employ non-linearities
- the building blocks of neural networks
 1. units
 2. layers
 3. implemented using matrix operations
- pros and cons of neural networks

Reading**Be prepared for substantial hybrid assignment for next week**

<https://www.deeplearningbook.org/contents/mlp.html>, Ch. 6: Deep Feedforward Networks

A Learning Algorithm for Boltzmann Machines*

DAVID H. ACKLEY
GEOFFREY E. HINTON

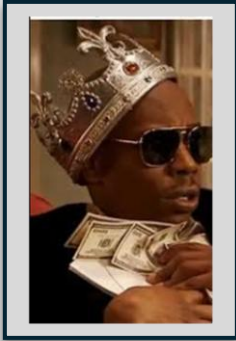
*Computer Science Department
Carnegie-Mellon University*

TERRENCE J. SEJNOWSKI

*Biophysics Department
The Johns Hopkins University*

Artificial Neural Networks

Logistic Regression (review)

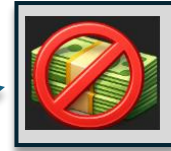


<https://tenor.com/search/im-rich-bitch-g>

$$\Pr(Y = 1 \mid X = x) = (1 + \exp(-x^T \beta))^{-1}$$

- logistic regression

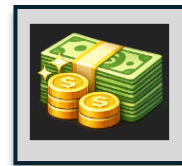
- it's **statistics**!
- it has been around **60 years**
- probably will **not** get you



$$\Pr[Y = 1 \mid X = x] = (1 + \exp(-f(x)^T \beta))^{-1}$$

- logistic regression with **neural network learned features**

- not statistics...**
- IS deep learning**
- has **ONLY** been around **50 years**
- GET PAID.**



Images by
ChatGPT 5.2

Neural Networks Big Picture

Machine Learning Supervised Loss

- supervised loss**: a set of **parameters** that are typically **operating linearly** over **input features**

$$\operatorname{argmin}_{\theta} \sum_{i=1}^N \ell_{\theta}(x_i, y_i)$$

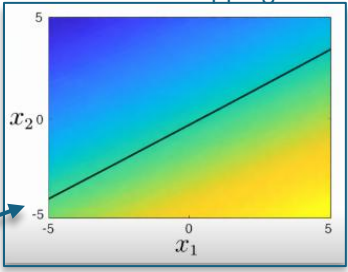
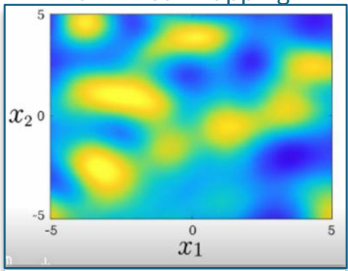
NN Supervised Loss

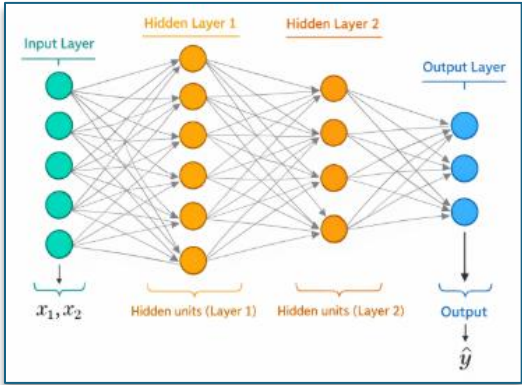
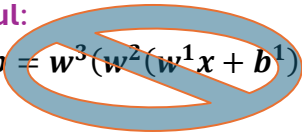
- still using a **linear** or similar **sigmoid predictor** over **learnable features**

$$\operatorname{argmin}_{\theta, w} \sum_{i=1}^N \ell_{\theta}(f_w(x_i), y_i)$$

- there are **additional parameters** known as **weights** (w) that we **optimize** over in **addition** to the linear **features**
- the weights determine the learnable features**
- the **number of output features** will **not** necessarily **coincide** with the **number of input features**

$$f_w(x): \mathbb{R}^d \rightarrow \mathbb{R}^k$$

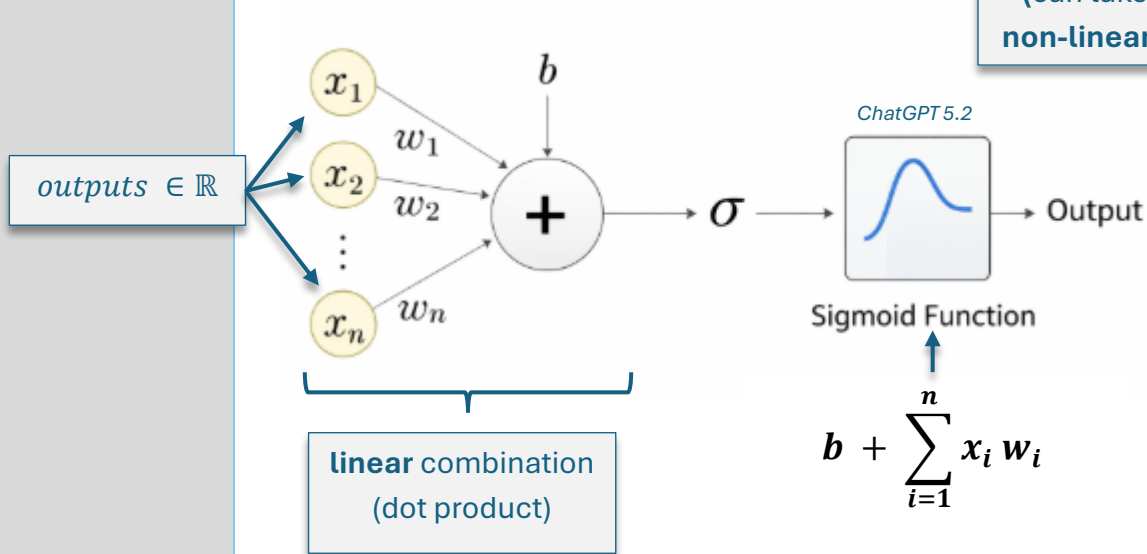
NN Regression Loss	$\operatorname{argmin}_{\theta, w} \sum_{i=1}^N \theta^T (f_w(x_i) - y_i)^2$ - key ingredients nonlinear - hierarchical universal
Non-Linear Mappings	
Linear Mapping	$\hat{y}_{\theta}(x_1, x_2) = \theta_1 x_1 + \theta_2 x_2 + \theta_0$ - many standard learning methods, such as ordinary least squares regression and logistic regression, are formulated as a linear map in the input space - this is adequate your goal is to map a boundary line between classes  ChatGPT 5.2
Non-Linear Mapping	$\hat{y}_{\theta}(x_1, x_2) = \sum_{j=1}^k \theta_j f_j(x_1, x_2)$ - these methods can be extended to a linear map in a nonlinear space - one perspective of what neural networks are doing is that they are learning a set of k features that can be linearly combined calculate an output  ChatGPT 5.2
Hierarchal Features	
“Stacking” Features	- one aspect that add to a neural network’s usefulness is that we can build features on top of features iteratively forming a hierarchical feature map - we can combine simple features to derive more complicated feature sets - this adds rich semantic meaning into calculated output features

One Layer	a single layer transforms input features to output features $\mathbb{R}^s \xrightarrow{\quad} \mathbb{R}^{s'}$ $x \Rightarrow x'$ $\text{input features} \Rightarrow \text{output features}$ the dimensions of the input vector does not have to match the dimensions of the output vector
Two Layers	in a two-layer NN, the output from the previous layer becomes the input for the next layer $x \Rightarrow x' \Rightarrow x''$ $\text{input features} \Rightarrow \text{output features} \Rightarrow \text{output features}$ Two-Layer Neural Network  ChatGPT 5.2
Features of Features	- this builds a function of nested functions $f_w(x) = f_{w_m}^m (\dots f_{w_2}^2 (f_{w_1}^1(x)) \dots)$ alternate notation $f_w(x) = (f_{w_m}^m \circ \dots \circ f_{w_2}^2 \circ f_{w_1}^1)(x)$
Building Blocks	
Function Composition	- simply stacking linear functions can become problematic - e.g. combining nested linear equations that result in a linear equation is not helpful: $f = wx + b = w^3(w^2(w^1x + b^1) + b^2) + b^3$ 

- the “fix” is to **add an addition function** which we call a **non-linearity** σ that takes the **output of the linear function** to **apply an element-wise non-linearity** as shown in the previous section:

$$(f_{w_m}^m \circ \dots \circ f_{w_2}^2 \circ f_{w_1}^1)(x) \quad \checkmark$$

(can take other
non-linear forms)



Matrix Notation

Layer $\ell - 1 \rightarrow \text{Layer } \ell$

$$\begin{matrix} \in \mathbb{R}^{s(\ell) \times s(\ell-1)} & \nearrow & \\ \varphi^{(\ell)} = \sigma(W^\ell \bar{\varphi}^{(\ell-1)} + \bar{b}^{(\ell)}) & & \\ \uparrow & \uparrow & \uparrow \\ \in \mathbb{R}^{s(\ell)} & \in \mathbb{R}^{s(\ell-1)} & \in \mathbb{R}^{s(\ell)} \end{matrix}$$

expanded form:

$$\begin{bmatrix} \varphi_1^\ell \\ \varphi_2^\ell \\ \vdots \end{bmatrix} = \sigma \left(\begin{bmatrix} -w_1^\ell \\ -w_2^\ell \\ \vdots \end{bmatrix} \begin{bmatrix} \bar{\varphi}_1^{(\ell-1)} \\ \bar{\varphi}_2^{(\ell-1)} \\ \vdots \end{bmatrix} + \begin{bmatrix} b_1^\ell \\ b_2^\ell \\ \vdots \end{bmatrix} \right) = \begin{bmatrix} \sigma(W^\ell(\bar{\varphi}^{(\ell-1)} + b_1^\ell)) \\ \vdots \\ \vdots \end{bmatrix}$$

network with one hidden layer

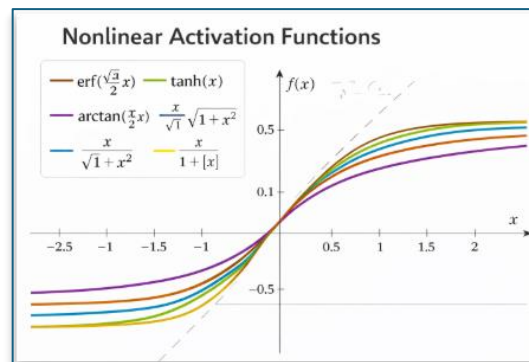
$$\sigma(W^{(1)}x + \bar{b}^{(1)})$$

regression loss for network with two hidden layers
and a linear output layer

$$(f(x) - y)^2 = (W^{(3)}\sigma(W^{(2)}\sigma(W^{(1)}x + \bar{b}^{(1)}) + \bar{b}^{(2)}) + \bar{b}^{(3)} - y^2)^2$$

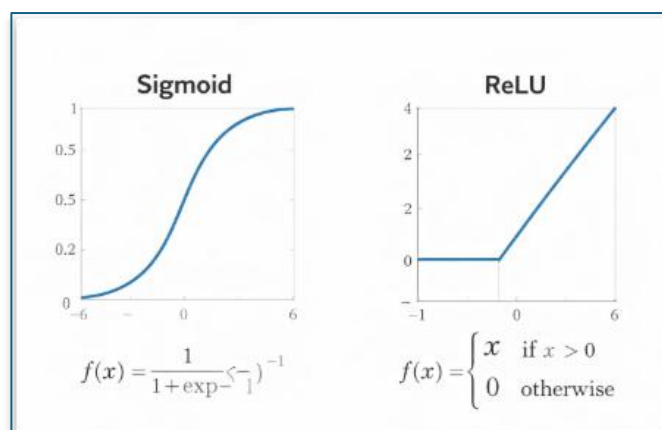
Non-linearities

- when we **compute gradients**, we multiply by the **derivatives of the activation functions**
- for **saturated units**—those receiving very **large positive** or very **large negative** inputs—the **slope** of the **activation function** is **close to zero**
- because the **derivative** is **near zero**, the **gradient** becomes very **small**
- as a result, **weight updates** during **gradient descent** are **minimal**, and learning **slows down** significantly

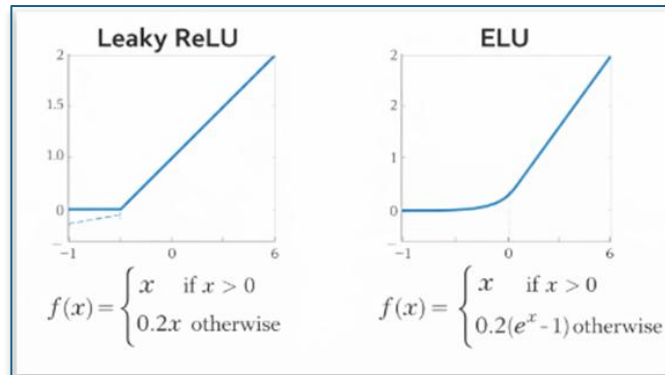


“vanishing gradients”

- to **resolve vanishing gradients**, non-linearities such as the **ReLU** or **rectified linear unit** have been used
- although the **ReLU** is a **simple** function, it is **non-linear** (piece-wise linear)
- **other** non-linear **functions** such as **Leaky ReLU** (ReLU with **non-zero** resolution of **negative** values) and **other piece-wise functions**



ChatGPT 5.2

*ChatGPT 5.2*

Live Session Notes

16 Feb 2026

- **Be prepared for substantial hybrid assignment for next week**

- HW02 is due 18 Feb
 - review BCE for this assignment
 - attn: data mining and machine learning. pg 623

$$LOSS = neg(log\ likelihood)E_{0,1}, L(\theta)$$

$$- \prod_i p(y_i | x_i, \theta)$$

$$logL(\theta) = E_i logP(y_i | x_i, \theta)$$

$$min - loss \leftrightarrow min (neg(log - likelihood)) \\ = neg(max (log - likelihood))$$

for a single sample with true label y E ???

and predicted probability p

if y=1: we want max(P(y=1|x))=p

if y=0: we want max(P(y=0|x)-1-p

BINARY CROSS ENTROPY (BCE)

Mathematical explanation

To get from the Binary Cross-Entropy formula to the Softplus version in the code, we substitute the sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

into the loss equation and simplify.

```
A = logits
B = -tf.math.softplus(-A)
mll = np.mean(Y * B + (1 - Y) * (-A + B))
```

Where,

```
logits = tf.matmul(X_with_bias, beta)
```

'one could also use'

- <https://playground.tensorflow.org/#activation=tanh&batchSize=10&dataset=circle®Dataset=reg-lane&learningRate=0.03®ularizationRate=0&noise=0&networkShape=4,2&seed=0.02680&showTestData=false&discretize=false&percTrainData=50&x=true&y=true&xTimesY=false&xSquared=false&ySquared=false&cosX=false&sinX=false&cosY=false&sinY=false&collectStats=false&problem=classification&initZero=false&hideText=false>
- The [Adam \(Adaptive Moment Estimation\) optimizer](#) is a popular, efficient gradient-based optimization algorithm used to train deep learning models by combining the advantages of [Momentum](#) and [RMSprop](#).

General Steps to Build a Neural Network: TensorFlow

Step Description

1. Import Libraries	Import TensorFlow and helper packages.
2. Prepare the Data	Load, clean, normalize, and split into train/test sets.
3. Define the Model	Use Sequential or Functional API (e.g., Input → Dense).
4. Compile the Model	Set optimizer, loss function, and metrics.
5. Train the Model	Use model.fit() with epochs and batch size.
6. Evaluate the Model	Use model.evaluate() on test data.
7. Make Predictions	Use model.predict() on new/unseen data.

- (comprehensive!) **project** proposal **feedback** has been sent out
 - **suggested references** have been **provided**
 - when perusing suggested references (**Google Scholar**), **check** at least one **paper cited** by that publication
 - focus on the methodology and reporting techniques
- be sure to mark the ‘completed by’ on CW assignments
 - coming up
 - week 7
 - neural networks 2
 - week 8
 - cross-validation
 - *midway report*
 - four page .pdf
 - NerulPS-style files w/ overleaf template
 - **MIDTERM!!!**

part 1

- questionnaire
 - multiple choice
 - free response
- study guide provided in week 7
- closed notes
- “optional questions” (extra credit?)

part 2

- applied problems
 - **long notebook**
 - **short sections**
 - **NO AGENTS!**